Empirical Evaluation of Kubernetes Horizontal Pod Autoscaling Effectiveness Under Dynamic Cloud Workloads in a Managed Cloud Environment (GKE)

Course: Cloud Computing

Instructor: Dr. Ming-Hwa Wang

Department of Computer Engineering

Santa Clara University

Team Members: Sriram Madduri, Kehan Chen, Lauren Hu

Date: February 23, 2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to Dr. Ming-Hwa Wang for his guidance, project structure, and emphasis on research-oriented learning in the Cloud Computing course. His project framework encourages students to approach engineering problems through a formal research methodology, which has greatly influenced the design and structure of this proposal.

We also acknowledge the support of Santa Clara University's academic resources and cloud computing learning environment, which provided the foundational knowledge in distributed systems, virtualization, and cloud-native architectures necessary to conceptualize this project.

Additionally, we thank the open-source community behind Kubernetes, Prometheus, and cloud-native tooling ecosystems for providing robust platforms that make empirical cloud research both accessible and reproducible. Their contributions to scalable infrastructure and observability tools make it possible to conduct meaningful experimental evaluations in real-world cloud environments such as Google Kubernetes Engine (GKE).

Finally, we appreciate the collaborative efforts of our team members, Lauren Hu, Sriram Madduri, and Kehan Chen, whose combined technical backgrounds in cloud computing, distributed systems, and software engineering contribute to the feasibility and rigor of this proposed research study.

TABLE OF CONTENTS

1. Title Page
2. Acknowledgements
3. Table of Contents
4. List of Figures and Tables
5. Abstract
6. Introduction
    - 6.1 Objective
    - 6.2 Problem Definition
    - 6.3 Use Cases
    - 6.4 Relevance to Cloud Computing
    - 6.5 Limitations of Existing Approaches
    - 6.6 Scope of Investigation
7. Theoretical Bases and Literature Review
    - 7.1 Definition of the Problem
    - 7.2 Theoretical Background of Kubernetes and Autoscaling
    - 7.3 Related Research
    - 7.4 Advantages and Disadvantages of Existing Research
    - 7.5 Proposed Solution and Novelty
8. Hypothesis
9. Methodology
    - 9.1 System Architecture

- 9.2 Data Collection Strategy
- 9.3 Workload Generation
- 9.4 Experimental Design
- 9.5 Performance Metrics
- 9.6 Cost Analysis Framework
- 9.7 Deployment Design (GKE)
- 9.8 Testing Against Hypotheses

10. Bibliography

LIST OF FIGURES AND TABLES

Figure 1: Overall System Architecture of HPA Evaluation Framework on GKE

Figure 2: Experimental Pipeline with Prometheus Monitoring and Locust Load Generation

Figure 3: Comparison Model – Fixed Deployment vs HPA Deployment

Figure 4: Data Collection Flow for Performance and Cost Metric

Table 1: Performance Metrics Definition and Measurement Method

Table 2: Experimental Configuration Parameters

Table 3: Baseline vs HPA Scaling Configuration

ABSTRACT

This proposal presents a research-driven empirical study that investigates whether Kubernetes Horizontal Pod Autoscaling (HPA) actually improves system performance under dynamic cloud workloads compared to a fixed replica deployment in a managed cloud environment. With the rapid adoption of cloud-native architectures and container orchestration platforms, autoscaling mechanisms have become a critical component in modern distributed systems. However, despite widespread industry use, there remains a need for systematic and quantitative evaluation of HPA effectiveness under realistic workload conditions.

The proposed project will deploy a Python-based web application on Google Kubernetes Engine (GKE) and conduct controlled load testing using Locust to simulate dynamic user traffic. Performance will be monitored using Prometheus and Kubernetes Metrics Server, enabling the collection of comprehensive metrics including latency, throughput, CPU utilization, scaling responsiveness, and cost efficiency. A fixed deployment baseline with three replicas will be compared against an HPA-enabled deployment configured to scale between one and ten replicas based on CPU utilization thresholds.

The methodology emphasizes reproducibility, measurable evaluation, and alignment with cloud computing principles such as elasticity, scalability, and resource efficiency. The results of this study aim to provide empirical evidence on whether autoscaling mechanisms deliver meaningful performance benefits in managed cloud environments or if static provisioning remains competitive under certain workload conditions.

INTRODUCTION

2.1 Objective

The primary objective of this project is to empirically evaluate the effectiveness of Kubernetes Horizontal Pod Autoscaling (HPA) in improving application performance under dynamic cloud workloads within a managed cloud environment, specifically Google Kubernetes Engine (GKE). While autoscaling is widely promoted as a core advantage of cloud-native systems, its actual performance impact under realistic and fluctuating workloads remains insufficiently quantified in academic settings.

This project seeks to systematically compare an HPA-enabled deployment against a fixed replica deployment using measurable cloud performance metrics such as response latency, throughput, CPU utilization, scaling responsiveness, and cost efficiency. By conducting controlled experiments in a real cloud infrastructure environment, this research aims to bridge the gap between theoretical cloud elasticity and practical system performance outcomes.
Furthermore, this project aligns directly with the core learning objectives of a Cloud Computing course, including resource elasticity, distributed workload management, scalability, and performance optimization in cloud-native systems.

2.2 What is the Problem

Modern cloud applications experience highly dynamic workloads, where user demand fluctuates unpredictably over time. Traditional fixed deployment models allocate a static number of server

instances (or pods) regardless of real-time traffic conditions. This often leads to two major issues: resource underutilization during low traffic periods and performance degradation during traffic spikes.

Kubernetes Horizontal Pod Autoscaling (HPA) was introduced as a solution to this problem by automatically adjusting the number of running pods based on observed resource metrics such as CPU utilization. In theory, HPA enables elastic scaling, which should improve system responsiveness and resource efficiency. However, several practical questions remain unanswered:

- Does HPA actually reduce latency under high load?
- How quickly does HPA react to sudden workload spikes?
- Is autoscaling more cost-efficient than static provisioning?
- Are there abnormal cases where HPA performs worse than fixed deployments?

These unanswered questions highlight the need for a structured empirical evaluation rather than relying solely on theoretical assumptions about autoscaling benefits.

2.3 Examples to Show the Problem and Use Cases

One common real-world example is a web application hosted on a cloud platform that experiences sudden increases in user traffic during peak hours. If the system uses a fixed deployment with three replicas, a traffic surge may overload the existing pods, resulting in increased response latency and degraded user experience.

In contrast, an HPA-enabled system can automatically scale the number of pods from three to a higher number, such as eight or ten, to handle the increased workload. This dynamic scaling capability is particularly important for:

- E-commerce platforms during sales events
- Streaming services during peak viewing times
- API-based microservices with burst traffic
- Cloud-hosted web applications with global users

However, autoscaling is not always guaranteed to be optimal. If scaling is too slow or misconfigured, performance may still degrade during traffic spikes. Additionally, excessive scaling may increase cloud costs unnecessarily. Therefore, a scientific evaluation is required to determine the true effectiveness of HPA in controlled cloud environments.

2.4 Why This is a Project Related to Cloud Computing

This project is fundamentally rooted in cloud computing concepts, particularly elasticity, scalability, resource management, and distributed system performance. Cloud computing platforms such as Google Cloud, AWS, and Azure emphasize on-demand resource provisioning as a core advantage over traditional on-premise systems.

Kubernetes, as a container orchestration platform, represents a key technology in cloud-native computing. Horizontal Pod Autoscaling is one of its most important features because it directly implements the cloud principle of automatic elasticity. By evaluating HPA in Google Kubernetes

Engine (GKE), this project explores how managed cloud services support scalable infrastructure and automated resource allocation.

Additionally, the integration of Prometheus monitoring and real-time metrics collection aligns with cloud observability practices, which are essential for modern distributed applications. The project also incorporates cost-performance tradeoff analysis, a critical topic in cloud computing where resource usage directly impacts operational expenses.

Thus, the proposed research strongly aligns with cloud computing curriculum topics including virtualization, container orchestration, elastic scaling, performance optimization, and cloud resource economics.

2.5 Why Other Approaches Are Not Good Enough

Traditional performance evaluation approaches often rely on synthetic benchmarks or theoretical modeling rather than real cloud deployments. While simulations can provide insights, they fail to capture real-world cloud infrastructure behavior such as network latency, container startup delays, and cloud scheduler dynamics.

Another limitation of existing studies is the lack of comprehensive metric evaluation. Many evaluations focus only on CPU utilization or throughput while ignoring important factors such as scaling latency and cost efficiency. Furthermore, some prior research uses simplified local Kubernetes clusters instead of managed cloud environments like GKE, which reduces the realism of the results.

Static provisioning approaches are also insufficient because they assume predictable workloads, which contradicts the dynamic nature of modern cloud applications. Over-provisioning wastes cloud resources and increases costs, while under-provisioning degrades performance and user experience.

Therefore, a controlled, real-cloud, metric-driven empirical study is necessary to accurately assess the advantages and limitations of Kubernetes Horizontal Pod Autoscaling.

2.6 Why We Think Our Approach is Better

Our proposed approach improves upon existing evaluation methods by combining real cloud deployment, dynamic workload simulation, comprehensive metric collection, and cost analysis within a single experimental framework.

Unlike simplified simulations, this project will deploy a real Python-based web application on Google Kubernetes Engine (GKE), ensuring that the results reflect actual cloud infrastructure behavior. The use of Prometheus for monitoring enables precise metric collection, while Locust-based workload generation provides reproducible and scalable traffic patterns. Additionally, our study introduces a direct baseline comparison between fixed deployment (static replicas) and HPA-enabled deployment under identical workload conditions. This controlled experimental design allows for fair and scientifically valid performance comparisons.

Another key advantage of our approach is the inclusion of cost efficiency analysis, which is often neglected in academic evaluations but is highly relevant in practical cloud computing scenarios.

By integrating performance metrics with cloud resource usage data, this project provides a holistic evaluation of autoscaling effectiveness.

THEORETICAL BASES AND LITERATURE REVIEW

3.1 Definition of the Problem

From a theoretical standpoint, the problem addressed in this project can be defined as an elastic resource allocation optimization problem in a cloud-native distributed system. Specifically, the system must dynamically adjust computational resources (pods) in response to time-varying workload demand while maintaining performance and cost efficiency.

Let W(t) represent the incoming workload at time ttt, measured in requests per second. Let R(t) represent the number of active replicas (pods) at time ttt, and let P(t) represent the performance metric (e.g., latency or throughput). In a fixed deployment model, R(t)=CR(t) = CR(t)=C, where C is a constant number of replicas. In contrast, an HPA-enabled deployment dynamically adjusts R(t) based on observed system metrics such as CPU utilization.

The core research problem can therefore be formalized as:

Does a dynamic scaling function $R(t) = f(\text{CPU}, W(t))$ provided by Kubernetes Horizontal Pod Autoscaling produce statistically significant improvements in performance metrics P(t) compared to a constant replica function R(t)=CR(t) = CR(t)=C under dynamic workloads?

Additionally, cloud computing environments introduce cost considerations. Let Cost(t) denote the cloud resource cost associated with running replicas over time. The optimization challenge is

not only to improve performance but also to balance the tradeoff between performance gains and operational costs.

Thus, the project evaluates both:

- Performance optimization (latency, throughput, responsiveness)
- Resource efficiency and cost optimization

This dual-objective perspective makes the problem highly relevant to real-world cloud computing systems.

3.2 Theoretical Background of Cloud Computing and Elastic Scaling

Cloud computing is fundamentally built upon the principles of scalability, elasticity, and on-demand resource provisioning. According to cloud computing theory, elasticity refers to the system's ability to dynamically allocate and deallocate computational resources based on workload demand. This capability distinguishes cloud platforms from traditional static infrastructure systems.

Container orchestration platforms such as Kubernetes play a critical role in enabling elasticity at the application layer. Kubernetes manages containerized workloads by scheduling pods across nodes, ensuring high availability, and supporting automated scaling mechanisms. Among its scaling features, Horizontal Pod Autoscaling (HPA) is one of the most widely used approaches for dynamic workload management.

The theoretical foundation of HPA is based on feedback control systems. In this model, the system continuously monitors resource utilization metrics (such as CPU usage) and compares them against a predefined target threshold. If the observed metric exceeds the threshold, the controller increases the number of replicas; if the metric falls below the threshold, it decreases the replicas. This creates a closed-loop control system aimed at maintaining system stability and performance.

From a distributed systems perspective, autoscaling also relates to load balancing and parallel processing theory. Increasing the number of replicas allows workload distribution across multiple processing units, theoretically reducing queueing delay and improving response time. However, real-world systems introduce overheads such as pod startup latency, scheduling delays, and network overhead, which may limit the theoretical performance gains.

3.3 Kubernetes Architecture and Horizontal Pod Autoscaling Theory

Kubernetes is a container orchestration platform designed to automate deployment, scaling, and management of containerized applications. Its architecture consists of a control plane and worker nodes. The control plane includes components such as the API server, scheduler, and controller manager, which collectively manage the lifecycle of pods and cluster resources.

Horizontal Pod Autoscaling operates as a controller within the Kubernetes control plane. It periodically retrieves metrics from the Kubernetes Metrics Server or external monitoring systems such as Prometheus. Based on these metrics, HPA calculates the desired number of replicas using the formula:

$$\text{Desired Replicas} = \lceil \text{Current Replicas} \times \frac{\text{Current Metric Value}}{\text{Target Metric Value}} \rceil$$

In most practical implementations, CPU utilization is used as the primary scaling metric. For example, if the target CPU utilization is set to 60% and the observed utilization reaches 90%, the HPA controller increases the number of replicas proportionally.

However, several theoretical limitations exist in this scaling mechanism. First, scaling decisions are reactive rather than predictive, meaning the system responds only after load increases. Second, scaling granularity is constrained by pod startup time, which can introduce delays during sudden workload spikes. Third, improper configuration of scaling thresholds can lead to oscillations, also known as “thrashing,” where the system repeatedly scales up and down inefficiently.

These theoretical considerations highlight the importance of empirical evaluation in real cloud environments such as Google Kubernetes Engine (GKE).

3.4 Theoretical Background of Observability and Performance Monitoring

In cloud-native systems, observability refers to the ability to measure and understand system behavior through metrics, logs, and traces. Accurate performance evaluation requires reliable metric collection and monitoring infrastructure.

Prometheus is a widely adopted open-source monitoring system designed for cloud-native environments. It follows a pull-based metric scraping model, where it periodically collects metrics from instrumented applications and infrastructure components. In Kubernetes environments, Prometheus can collect metrics related to CPU usage, memory consumption, request latency, and pod scaling events.

The integration of Prometheus with Kubernetes provides a robust framework for empirical performance analysis. By storing time-series metrics, researchers can analyze workload patterns, scaling responsiveness, and system stability over time. This is particularly important for evaluating autoscaling mechanisms, as performance improvements must be measured quantitatively rather than subjectively.

Furthermore, observability tools enable abnormal case analysis, which is a critical requirement in research-oriented projects. For example, if autoscaling fails to respond quickly to traffic spikes, metric logs can help identify bottlenecks such as delayed metric collection or slow pod initialization.

3.5 Related Research on Autoscaling in Cloud Environments

A significant body of research has explored autoscaling mechanisms in cloud computing systems. Early studies on cloud elasticity focused on virtual machine scaling, where infrastructure resources were dynamically provisioned based on workload demand. With the rise of containerization, more recent research has shifted toward container-level autoscaling using orchestration platforms such as Kubernetes.

Several academic studies suggest that autoscaling can significantly improve system responsiveness under fluctuating workloads by distributing requests across multiple replicas. Research on microservices architectures indicates that horizontal scaling reduces bottlenecks and enhances fault tolerance, particularly in web-based applications with burst traffic patterns. Industry research and cloud provider documentation also emphasize the benefits of autoscaling in managed environments such as Google Kubernetes Engine (GKE), Amazon EKS, and Azure AKS. These platforms provide built-in autoscaling mechanisms designed to optimize resource utilization and reduce operational costs.

However, not all studies report universally positive outcomes. Some research highlights that autoscaling performance depends heavily on workload characteristics, scaling thresholds, and monitoring accuracy. For instance, sudden workload spikes may cause temporary performance degradation due to delayed scaling reactions. Additionally, over-aggressive scaling policies may lead to excessive resource provisioning and increased cloud costs.
These mixed findings indicate that autoscaling effectiveness is context-dependent, reinforcing the need for controlled experimental evaluation.

3.6 Advantages and Disadvantages of Existing Research

Existing research on autoscaling provides valuable insights but also exhibits several limitations. One major advantage is that prior studies establish the theoretical importance of elasticity in cloud computing systems. They demonstrate that dynamic scaling can improve throughput and system availability under variable workloads.

Another strength of existing research is the adoption of real-world benchmarks and cloud-native applications, which enhance the practical relevance of experimental results. Many studies also utilize monitoring frameworks similar to Prometheus, enabling detailed metric analysis.
Despite these advantages, several gaps remain. First, many studies focus on simulation environments or simplified local clusters rather than fully managed cloud platforms like GKE. This reduces the realism of their findings. Second, some evaluations consider only a limited set of metrics, such as CPU utilization or throughput, without analyzing latency distributions or cost efficiency.

Additionally, few studies conduct direct comparisons between fixed deployment baselines and HPA-enabled systems under identical experimental conditions. Without a controlled baseline, it is difficult to determine whether performance improvements are truly due to autoscaling or other system factors.

3.7 Our Solution to Solve This Problem

To address the limitations identified in prior research, this project proposes a comprehensive empirical evaluation framework that directly compares Kubernetes Horizontal Pod Autoscaling against a fixed replica deployment in a managed cloud environment (GKE).
Our solution integrates four key components:

1. A Python-based web application to simulate realistic cloud workloads
2. Google Kubernetes Engine for real cloud deployment
3. Prometheus for high-resolution metric monitoring

4. Locust for reproducible workload generation

By combining these components, the proposed framework enables controlled experimentation with dynamic workloads while maintaining high reproducibility and measurement accuracy.

Unlike simplified benchmark approaches, this project evaluates autoscaling performance in a real cloud infrastructure setting, capturing practical factors such as network latency, container startup time, and cloud scheduler behavior.

3.8 Where Our Solution is Different from Others

Our approach differs from existing research in several important ways. First, it focuses specifically on managed cloud environments (GKE) rather than local clusters or simulations, which improves the external validity of the results.

Second, the study incorporates a multi-metric evaluation framework that includes latency percentiles (p50, p95, p99), throughput, CPU utilization, scaling responsiveness, and cost efficiency. This holistic evaluation provides deeper insights compared to studies that rely on a single metric.

Third, the project introduces a clearly defined baseline comparison using a fixed three-replica deployment, ensuring fair and scientifically controlled experimentation. Many prior studies lack such controlled baselines, making their conclusions less definitive.

Finally, the inclusion of cost-performance tradeoff analysis distinguishes this research from purely performance-focused studies. In real cloud computing environments, cost is a critical factor, and evaluating autoscaling without cost considerations provides an incomplete assessment.

3.9 Why Our Solution is Better

Our proposed solution is better because it combines theoretical rigor, real-world deployment, comprehensive metrics, and cost analysis into a unified research framework. It directly addresses the key research gap of insufficient empirical evaluation of Kubernetes HPA in managed cloud environments.

By using GKE, Prometheus, and controlled load generation, the study ensures high experimental validity and reproducibility. The clear baseline comparison eliminates ambiguity in performance evaluation, while the inclusion of abnormal case analysis strengthens the academic quality of the research.

Moreover, the clarity and transparency of the methodology make the results easier to interpret, present, and validate, which is especially important in a Cloud Computing academic setting.

HYPOTHESIS

4.1 Overview of Hypothesis Formulation

In a research-oriented cloud computing project, hypotheses serve as formal statements that define the expected outcomes of the experimental study. Instead of simply stating goals, hypotheses provide measurable and testable claims that can be validated through empirical data collection and analysis. This aligns with the scientific research methodology emphasized in advanced cloud computing and distributed systems studies.

The central focus of this project is to determine whether Kubernetes Horizontal Pod Autoscaling (HPA) provides measurable performance and cost benefits compared to a fixed deployment model under dynamic cloud workloads in a managed cloud environment (Google Kubernetes Engine). Therefore, the hypotheses are formulated to evaluate both performance effectiveness and resource efficiency using quantitative cloud metrics.

To ensure clarity and academic rigor, this project adopts multiple positive and evaluative hypotheses that can be directly tested using experimental results collected through Prometheus monitoring, Locust workload generation, and Kubernetes metrics.

4.2 Primary Hypothesis

H_1 (Primary Hypothesis – Positive):

Kubernetes Horizontal Pod Autoscaling (HPA) will significantly improve application

performance under dynamic workloads compared to a fixed deployment with static replicas in a managed cloud environment (GKE).

This hypothesis is based on the theoretical principle of cloud elasticity, which states that dynamic resource scaling should enhance system responsiveness when workload demand fluctuates. Specifically, HPA is expected to reduce response latency and maintain higher throughput during traffic spikes by automatically increasing the number of running pods.

The primary performance indicators used to validate this hypothesis include:

- Response latency (p50, p95, p99)
- Throughput (requests per second)
- System stability under load

If the experimental results show consistently lower latency and higher throughput in the HPA deployment compared to the fixed deployment under identical workload conditions, this hypothesis will be supported.

4.3 Secondary Hypothesis

H_2 (Secondary Hypothesis – Positive):

An HPA-enabled deployment will demonstrate superior scalability and faster adaptation to workload fluctuations compared to a fixed replica deployment.

This hypothesis focuses on the responsiveness of the autoscaling mechanism. In dynamic cloud environments, workloads often change rapidly, and systems must adapt in real time to maintain

service quality. Horizontal Pod Autoscaling uses CPU utilization metrics and control loops to adjust replica counts, which theoretically allows the system to scale resources according to demand.

The validation of this hypothesis will be based on:

- Pod scaling time (time required to scale up or down)
- Replica adjustment patterns during workload spikes
- CPU utilization stabilization over time

If the HPA deployment shows timely scaling behavior and maintains stable CPU utilization levels during fluctuating traffic, this hypothesis will be considered validated.

4.4 Tertiary Hypothesis

H_3 (Cost Efficiency Hypothesis – Positive):
Kubernetes Horizontal Pod Autoscaling will achieve better cost-performance efficiency compared to a fixed deployment by allocating cloud resources dynamically based on real-time workload demand.

Cloud computing environments operate on a pay-as-you-go pricing model, where resource usage directly impacts operational cost. A fixed deployment continuously consumes resources regardless of traffic demand, which may lead to unnecessary expenses during low workload periods.

In contrast, HPA can scale down the number of replicas during periods of low demand, potentially reducing compute resource consumption and associated cloud costs. However, scaling up during peak demand may temporarily increase costs. Therefore, the overall cost efficiency will be evaluated using metrics such as:

- Total compute resource usage (CPU and pod-hours)
- Cost per request
- Cost-performance ratio (cost vs throughput and latency)

If HPA demonstrates lower cost per request while maintaining acceptable performance levels, the cost efficiency hypothesis will be supported.

4.5 Null Hypothesis

H_0 (Null Hypothesis):

There is no statistically significant difference in performance and cost efficiency between an HPA-enabled deployment and a fixed deployment under dynamic cloud workloads.

The null hypothesis is essential for scientific validity because it provides a baseline assumption that autoscaling may not necessarily produce measurable improvements. In some cases, autoscaling overhead, delayed scaling reactions, or improper configuration may result in performance outcomes similar to or worse than fixed deployments.

Rejecting or failing to reject the null hypothesis will depend on the experimental data collected during controlled testing on Google Kubernetes Engine.

4.6 Negative Hypothesis

H_4 (Negative Hypothesis – Conditional):

Under sudden and extreme workload spikes, Kubernetes Horizontal Pod Autoscaling may temporarily underperform compared to a fixed deployment due to scaling latency and pod initialization overhead.

This hypothesis acknowledges realistic limitations of autoscaling systems. Since HPA relies on periodic metric collection and reactive scaling, there may be delays between workload increase and replica scaling. During this delay window, the system may experience higher latency compared to a pre-provisioned fixed deployment with sufficient replicas.

This hypothesis will be evaluated through abnormal case analysis, which is an important research component in empirical cloud system studies. Metrics such as scaling delay and transient latency spikes will be analyzed to determine whether HPA introduces short-term performance degradation during rapid traffic surges.

4.7 Hypothesis Testing Strategy

To test the proposed hypotheses, controlled experiments will be conducted using identical workloads across two deployment configurations:

1. Fixed Deployment (3 static replicas)
2. HPA Deployment (1–10 replicas, CPU target 60%)

The experiments will be executed on Google Kubernetes Engine to ensure realistic cloud infrastructure behavior. Locust will generate reproducible dynamic workloads, while Prometheus and Kubernetes Metrics Server will collect high-resolution performance and resource metrics. Statistical comparison methods will be used to evaluate:

- Mean and percentile latency differences
- Throughput variations under load
- Resource utilization trends
- Cost-performance tradeoffs

By analyzing these metrics across multiple experimental runs, the study will determine whether the observed results support or reject the stated hypotheses.

4.8 Expected Contributions Based on Hypotheses

If the primary and secondary hypotheses are validated, this project will provide empirical evidence that Kubernetes Horizontal Pod Autoscaling improves performance and scalability in managed cloud environments. If the cost hypothesis is also supported, the research will demonstrate that HPA not only enhances performance but also optimizes resource efficiency in cloud-native applications.

Even if certain hypotheses are only partially supported, the findings will still contribute valuable insights into the practical limitations of autoscaling mechanisms, particularly in terms of scaling latency, cost tradeoffs, and abnormal workload behavior.

This hypothesis-driven approach ensures that the project is not merely an implementation exercise but a structured cloud computing research study with measurable and academically rigorous evaluation criteria.

METHODOLOGY

5.1 Overview of Research Methodology

This project adopts an empirical, experimental, and reproducible research methodology to evaluate the effectiveness of Kubernetes Horizontal Pod Autoscaling (HPA) in a managed cloud environment. The methodology is designed to align with cloud computing research standards by incorporating real-world deployment, controlled workload generation, comprehensive metric collection, and cost-performance analysis.

Rather than relying on simulations or theoretical assumptions, the study will deploy a real Python-based web application on Google Kubernetes Engine (GKE) and subject it to dynamically generated workloads. The experimental framework will compare two deployment strategies: a fixed replica deployment and an HPA-enabled deployment under identical workload conditions.

The methodology emphasizes scientific validity through controlled variables, repeatable experiments, and quantitative evaluation using multiple performance and cost metrics. This structured approach ensures that the results can be objectively analyzed and directly mapped to the hypotheses defined in Part 4.

5.2 System Architecture Design

The overall system architecture consists of four primary layers:

1. Application Layer (Python Web Application)
2. Orchestration Layer (Kubernetes on GKE)
3. Monitoring Layer (Prometheus + Metrics Server)
4. Workload Generation Layer (Locust Load Testing)

In this architecture, a containerized Python FastAPI web application will be deployed to a Kubernetes cluster hosted on Google Kubernetes Engine. The Kubernetes control plane will manage pod scheduling, scaling, and resource allocation. Prometheus will continuously scrape performance metrics, while Locust will simulate dynamic user traffic patterns.

This layered architecture ensures modularity, scalability, and accurate performance observation in a real cloud-native environment. Additionally, it reflects modern industry practices in cloud-native system evaluation and observability.

5.3 Deployment Environment

The experimental platform will be Google Kubernetes Engine (GKE), a managed Kubernetes service that provides automated cluster management, scalability, and high availability. Using GKE enhances the realism of the study because it replicates real-world cloud deployment conditions, including container orchestration overhead, network latency, and cloud scheduler behavior.

The cluster configuration will follow a standard academic baseline design:

- Region: us-central1

- Node Type: e2-standard-2 (cost-efficient and commonly used in research)
- Number of Nodes: 2–3 worker nodes
- Kubernetes Version: Stable GKE release

Using a managed cloud environment rather than a local Kubernetes cluster ensures that the results reflect actual cloud computing behavior rather than simplified laboratory conditions.

5.4 Application Design

The application used in this study will be a lightweight Python-based web service developed using the FastAPI framework. This web application will expose REST API endpoints capable of handling concurrent HTTP requests and returning structured responses.

The rationale for selecting a web application workload is that web services represent one of the most common real-world cloud-native workloads. They experience fluctuating traffic patterns and require scalable infrastructure to maintain performance under dynamic demand.

The application will be containerized using Docker and deployed as Kubernetes pods. Resource requests and limits will be defined to ensure consistent CPU allocation across experiments. This design allows for fair comparison between fixed deployment and HPA configurations.

5.5 Data Collection and Input Generation

5.5.1 Workload Generation Using Locust

Dynamic workload input will be generated using Locust, a Python-based load testing tool that supports distributed and scalable traffic simulation. Locust is selected because it provides reproducible workload patterns, customizable user behavior models, and seamless integration with cloud-native environments.

The workload profile will include:

- Gradual traffic ramp-up phase
- Sudden workload spikes
- Sustained high-load periods
- Low-demand recovery phase

This dynamic workload model ensures that the autoscaling mechanism is evaluated under realistic cloud traffic scenarios rather than static or uniform request patterns.

5.5.2 Input Data Characteristics

The input data will consist of HTTP requests sent to the deployed web application endpoints. Each request simulates a real user interaction with the cloud-hosted service. Key characteristics of the input workload include:

- Concurrent virtual users
- Variable request frequency

- Randomized request intervals
- Burst traffic simulation

These characteristics are essential for testing how effectively HPA adapts to unpredictable cloud workloads.

5.6 Experimental Design and Baseline Configuration

5.6.1 Fixed Deployment

The baseline system will use a static Kubernetes deployment with a fixed number of replicas. For this study, the baseline will be configured with three replicas, which represents a standard academic and industry baseline for small-scale cloud services.

Key configuration parameters:

- Replicas: 3 (constant)
- No autoscaling enabled
- Identical resource limits as HPA deployment

This baseline ensures a fair comparison because both systems will run the same application under identical workload conditions.

5.6.2 HPA Deployment

The experimental system will enable Kubernetes Horizontal Pod Autoscaling with the following configuration:

- Minimum Replicas: 1
- Maximum Replicas: 10
- Target CPU Utilization: 60%
- Scaling Policy: Reactive scaling based on CPU metrics

This configuration is widely accepted in cloud research as a balanced autoscaling setup that avoids excessive scaling while maintaining responsiveness to workload changes.

5.7 Algorithm Design and Scaling Logic

Although Kubernetes HPA is managed by built-in controllers, the scaling logic follows a well-defined control algorithm. The system periodically monitors CPU utilization and calculates the desired number of replicas using a proportional scaling formula.

The scaling process can be summarized as:

1. Collect real-time CPU utilization metrics
2. Compare current utilization with target threshold
3. Compute desired replica count
4. Adjust pod replicas accordingly
5. Stabilize system performance through feedback loop

This feedback-based control mechanism allows the system to dynamically allocate resources in response to workload changes while maintaining system stability.

5.8 Monitoring and Observability Tools

5.8.1 Prometheus Monitoring System

Prometheus will serve as the primary monitoring and data collection tool. It will scrape metrics from Kubernetes nodes, pods, and application endpoints at regular intervals. The collected metrics will be stored as time-series data for detailed performance analysis.

Key monitored metrics include:

- CPU utilization per pod
- Memory usage
- Request latency
- Throughput (requests per second)
- Pod scaling events

5.8.2 Kubernetes Metrics Server

The Kubernetes Metrics Server will provide real-time resource usage data that is required for HPA decision-making. This ensures that autoscaling actions are based on accurate and up-to-date system metrics.

The combination of Prometheus and Metrics Server creates a robust observability framework for empirical cloud system evaluation.

5.9 Performance Metrics and Output Generation

To comprehensively evaluate autoscaling effectiveness, multiple quantitative performance metrics will be collected and analyzed. These include:

1. Latency Metrics
    - p50 (median latency)
    - p95 latency
    - p99 latency
2. Throughput
    - Number of requests processed per second
3. CPU Utilization
    - Average CPU usage per pod
    - Cluster-level CPU utilization
4. Scaling Responsiveness
    - Time required to scale up replicas
    - Time required to scale down replicas
5. System Stability
    - Error rates
    - Performance fluctuations during scaling events

The output data will be exported from Prometheus and analyzed using statistical methods and visualization tools such as Python (Matplotlib/Pandas).

5.10 Cost Analysis Framework

Since cloud environments operate on a pay-as-you-go pricing model, cost efficiency is a critical evaluation factor. This project will incorporate a cost-performance analysis to determine whether HPA provides economic benefits in addition to performance improvements.

Cost metrics will include:

- Compute resource usage (node hours and pod hours)
- Estimated GKE operational cost
- Cost per request processed
- Cost-performance ratio

By comparing resource consumption between fixed and autoscaling deployments, the study will quantify whether dynamic scaling reduces unnecessary resource allocation during low-demand periods.

5.11 Deployment Pipeline and Implementation Workflow

The deployment workflow will follow a structured cloud-native DevOps pipeline:

1. Develop and containerize the Python web application using Docker
2. Push container images to a container registry
3. Deploy application to GKE cluster using Kubernetes manifests
4. Configure HPA policies and resource limits
5. Install Prometheus and Metrics Server for monitoring
6. Execute Locust load testing experiments

7. Collect and store performance metrics
8. Analyze results and compare against hypotheses

This pipeline ensures reproducibility and systematic experimentation, which are essential for academic research projects.

5.12 How to Test Against Hypotheses

To validate the hypotheses defined in Part 4, multiple experimental trials will be conducted under identical workload conditions for both deployment models. Each experiment will be repeated several times to ensure statistical reliability and eliminate random variations.

The testing process will include:

- Running baseline (fixed deployment) experiments
- Running HPA-enabled experiments
- Collecting performance and cost metrics
- Performing comparative statistical analysis
- Identifying abnormal cases and anomalies

Hypothesis validation will be based on measurable differences in latency, throughput, scaling responsiveness, and cost efficiency. If the HPA deployment consistently outperforms the fixed deployment across these metrics, the primary and secondary hypotheses will be supported.

5.13 Abnormal Case Analysis

A key component of this methodology is the analysis of abnormal cases, such as delayed scaling reactions, sudden latency spikes, or inefficient scaling oscillations. These abnormal scenarios are particularly important in cloud computing research because real-world systems rarely behave ideally under extreme workloads.

By analyzing metric logs and scaling events, the study will identify situations where autoscaling may temporarily degrade performance due to reactive scaling delays or infrastructure overhead. This analysis strengthens the academic rigor of the project by acknowledging both strengths and limitations of HPA in practical cloud environments.

5.14 Summary of Methodology Strength

The proposed methodology is comprehensive, reproducible, and aligned with cloud computing research standards. It integrates real cloud deployment (GKE), dynamic workload simulation (Locust), advanced monitoring (Prometheus), and cost-performance evaluation into a unified experimental framework.

This methodological design ensures fair baseline comparison, accurate metric collection, and scientifically valid hypothesis testing. As a result, the study will produce reliable empirical evidence regarding the effectiveness of Kubernetes Horizontal Pod Autoscaling in modern cloud-native systems.

BIBLIOGRAPHY

9.1 Academic and Research References

1. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016).
   Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade.
   Communications of the ACM, 59(5), 50–57.
2. Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., & Wilkes, J. (2015).
   Large-scale cluster management at Google with Borg.
   Proceedings of the Tenth European Conference on Computer Systems (EuroSys).
3. Lorido-Botrán, T., Miguel-Alonso, J., & Lozano, J. A. (2014).
   A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments.
   Journal of Grid Computing, 12(4), 559–592.
4. Mao, M., & Humphrey, M. (2012).
   Auto-scaling to minimize cost and meet application deadlines in cloud workflows.
   Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC).
5. Herbst, N. R., Kounev, S., & Reussner, R. (2013).
   Elasticity in Cloud Computing: What It Is, and What It Is Not.
   Proceedings of the 10th International Conference on Autonomic Computing (ICAC).

6. Ali-Eldin, A., Tordsson, J., & Elmroth, E. (2012).
   An Adaptive Hybrid Elasticity Controller for Cloud Infrastructures.
   IEEE Network, 26(1), 10–16.
7. Calheiros, R. N., Ranjan, R., Beloglazov, A., De Rose, C. A., & Buyya, R. (2011).
   CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms.
   Software: Practice and Experience, 41(1), 23–50.
8. Pahl, C. (2015).
   Containerization and the PaaS Cloud.
   IEEE Cloud Computing, 2(3), 24–31.

9.2 Kubernetes and Cloud-Native Technology References

9. Hightower, K., Burns, B., & Beda, J. (2017).
   Kubernetes: Up and Running: Dive into the Future of Infrastructure.
   O'Reilly Media.
10. Kubernetes Documentation. (2024).
    Horizontal Pod Autoscaling.
    The Linux Foundation & Cloud Native Computing Foundation (CNCF).
11. Cloud Native Computing Foundation (CNCF). (2023).
    Cloud Native Definition v1.0.
    CNCF Whitepaper.

12. Merkel, D. (2014).
Docker: Lightweight Linux containers for consistent development and deployment.
Linux Journal, 2014(239).
13. Google Cloud. (2024).
Google Kubernetes Engine (GKE) Documentation.
Google Cloud Platform Technical Documentation.

9.3 Monitoring, Observability, and Performance Evaluation

References

14. Turnbull, J. (2018).
The Prometheus Monitoring System and Service Discovery.
O'Reilly Media.
15. Sigelman, B., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010).
Dapper, a Large-Scale Distributed Systems Tracing Infrastructure.
Google Research Technical Report.
16. Barham, P., et al. (2003).
Xen and the Art of Virtualization.
ACM Symposium on Operating Systems Principles (SOSP).
17. Jain, R. (1991).
The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling.
Wiley-Interscience.

9.4 Load Testing and Cloud Workload Research References

18. Grambow, M., Pfandzelter, T., & Bermbach, D. (2020).

    Benchmarking Elasticity of Cloud Applications.

    IEEE International Conference on Cloud Engineering (IC2E).

19. Vasić, N., Novaković, D., Mišić, J., & Mišić, V. B. (2012).

    DejaVu: Accelerating Resource Allocation in Virtualized Environments.

    ACM SIGPLAN Notices.

20. Locust.io Documentation. (2024).

    Locust: Scalable Load Testing Framework.

    Open Source Documentation.

9.5 Cost and Cloud Resource Optimization References

21. Armbrust, M., et al. (2010).

    A View of Cloud Computing.

    Communications of the ACM, 53(4), 50–58.

22. Gmach, D., Rolia, J., Cherkasova, L., & Kemper, A. (2007).

    Workload Analysis and Demand Prediction of Enterprise Data Center Applications.

    IEEE International Symposium on Workload Characterization.

23. Chaisiri, S., Lee, B. S., & Niyato, D. (2012).

    Optimization of Resource Provisioning Cost in Cloud Computing.

    IEEE Transactions on Services Computing, 5(2), 164–177.